

Bohrin on 20 Public LeRobot Datasets

A Precision Audit of 48 Data-Quality Detectors for Robot Demonstration Data

Prabhu Gopal

26 August 2026 · bohrin 0.1.0 · commit cf39d78

Abstract

Bohrin is a static analyzer for robot demonstration datasets: it reads logged states and actions and reports defects that are known to degrade a behavior-cloned policy. Its unit suite plants known defects in synthetic fixtures and verifies they are found, which measures *recall* but cannot, in principle, measure *precision on healthy data*. This report supplies the missing half. We scan 20 curated, published LeRobot datasets (2,631 episodes, 545,964 frames, 8 embodiment families, 2–40 action dimensions) with the default detector configuration and report how often each of the 46 default detectors complains. All 20 datasets parsed without error in 20.8s total on a laptop CPU, yielding 187 findings (23 HIGH, 119 MEDIUM, 45 LOW). Twenty-seven detectors fired at least once; nineteen never fired; only four ever reached HIGH. One detector, `dynamics.inverse_residual`, reports HIGH on 50% of the corpus and is flagged here as not yet trustworthy. We state explicitly what this measurement does *not* establish: without ground-truth labels, a fire rate is a rate of complaint, not a rate of correctness. A secondary finding concerns the ecosystem rather than the tool: 18 of 20 datasets declare `robot_type: "unknown"`, which undermines any embodiment-keyed calibration scheme built on public metadata.

1 Motivation

A data-quality detector is a claim that a specific pattern in the data predicts a specific training failure. Testing such a claim against a synthetic fixture — inject the defect, confirm the detector fires — establishes recall and nothing else. A detector hard-coded to return HIGH would pass every such test.

Precision is what determines whether the tool is usable. A HIGH severity that fires on most curated public datasets carries no information, and it breaks a `--fail-on HIGH` CI gate for every user. The only way to measure precision is to point the detectors at data that other people collected, curated, published, and trained on, and count the complaints. This report is that measurement.

2 Corpus and Method

Twenty LeRobot datasets were selected for breadth of embodiment and provenance rather than size: ALOHA (simulated and mobile), xArm, PR2, Franka, Stretch, Unitree H1, the DLR/NYU/UCSD/Tokyo laboratory rigs, and the `pusht` family. Action dimensionality spans 2 to 40; control rates span 5 Hz to 50 Hz; both simulated and real-robot data are represented. Bohrin does not decode video, so the full corpus is approximately 50 MB.

Each dataset was scanned with the default configuration: no policy checkpoint, no `--target`, no `--all`, no `--full`, no calibration corpus, and vision disabled. The harness is `scripts/hub_smoke.py` in the bohrin repository; raw output is committed as `results/sweep.json`, from which every figure and table here is derived programmatically.

3 Results

3.1 Coverage and cost

All 20 datasets parsed, profiled, and reported with no crash, no unhandled exception, and no adapter failure. Total wall-clock time was 20.8s on an Apple M4 laptop (16 GB, CPU only, warm HTTP cache):

Table 1: Corpus summary. All twenty datasets resolved as `lerobot_v3`. H/M/L are counts of HIGH/MEDIUM/LOW findings.

Dataset	Hz	act	state	Episodes	Frames	Scan (s)	H	M/L
xarm_lift_medium	15	4	4	300	20,000	0.9	1	4/2
ucsd_pick_and_place	5	4	7	300	67,750	1.2	2	8/3
nyu_franka_play	5	15	13	300	44,875	1.9	1	7/3
utokyo_pr2_tabletop	5	8	7	240	32,708	1.1	2	10/3
pusht	10	2	2	206	25,650	0.7	0	3/4
pusht_keypoints	10	2	2	206	25,650	0.9	0	3/4
ucsd_kitchen	5	8	21	150	3,970	0.6	1	7/2
cmu_stretch	5	8	4	135	25,016	0.8	3	8/1
asu_table_top	5	7	7	110	26,113	1.0	1	6/1
dlr_sara_grid_clamp	5	7	12	107	7,622	0.6	1	4/4
dlr_edan_shared_control	5	7	7	104	8,928	0.6	1	9/2
dlr_sara_pour	5	7	6	100	12,971	0.6	1	8/4
aloha_mobile_cabinet	50	14	14	85	127,500	2.1	1	5/1
utokyo_pr2_fridge	5	8	7	80	11,522	0.6	1	7/2
tokyo_u_lsмо	5	7	13	50	11,925	0.6	1	4/2
aloha_sim_insertion	50	14	14	50	25,000	0.9	1	3/1
austin_buds	5	7	24	50	34,112	1.0	2	6/2
unitreeh1_warehouse	50	40	19	24	11,275	0.8	0	6/2
utokyo_saytap	5	12	30	20	22,937	0.9	1	5/0
nyu_rot	5	7	7	14	440	3.0	2	6/2
Total				2,631	545,964	20.8	23	119/45

Table 2: The four detectors that ever reported HIGH.

Detector	Fires on	of which HIGH	Reading
<code>dynamics.inverse_residual</code>	95%	50%	Under suspicion (§3.3)
<code>stats.dead_dimension</code>	35%	35%	Plausible; dead dims are common
<code>label.trajectory_label_mismatch</code>	20%	20%	Plausible at this rate
<code>multimodality.contradictory_actions</code>	10%	10%	Plausible at this rate

median 0.9 s per dataset, range 0.6–3.0 s. Seventeen of twenty datasets produced at least one HIGH; none came back completely clean.

Figure 1 shows scan cost is effectively flat in episode count. `nyu_rot` is the slowest at 3.0 s despite being the smallest dataset in the corpus (14 episodes, 440 frames) — it was scanned first and absorbed one-time interpreter and `scikit-learn` import cost. `aloha_mobile_cabinet`, with 290× more frames, took 2.1 s. Cost tracks column reads and per-detector fixed overhead, not data volume.

3.2 Precision measurement

Of the 46 detectors that run in a default scan (48 registered, 2 held back; see Section 4), **27 fired at least once** and **19 never fired on any dataset**. Only **four** ever reached HIGH (Table 2, Figure 3).

3.3 `dynamics.inverse_residual` is the outstanding problem

This detector fires on 19 of 20 datasets and returns HIGH on 10 of 20. A detector that reports HIGH on half of a curated, published, widely used corpus is more likely to have a threshold defect than to have found an epidemic.

Two readings are consistent with the number, and this sweep cannot distinguish them. *(i) The threshold is loose:* the residual of a fitted inverse-dynamics model $g(o_t, o_{t+1}) \rightarrow \hat{a}_t$ is compared against a bound that ordinary sensor noise and 5 Hz sub-sampling routinely exceed — and most of this corpus is 5 Hz, slow enough that consecutive states are only weakly related by the logged action. *(ii) The finding is real:* action/observation misalignment is a common defect in converted datasets, and most of this corpus

Scan cost is set by column reads, not episode count

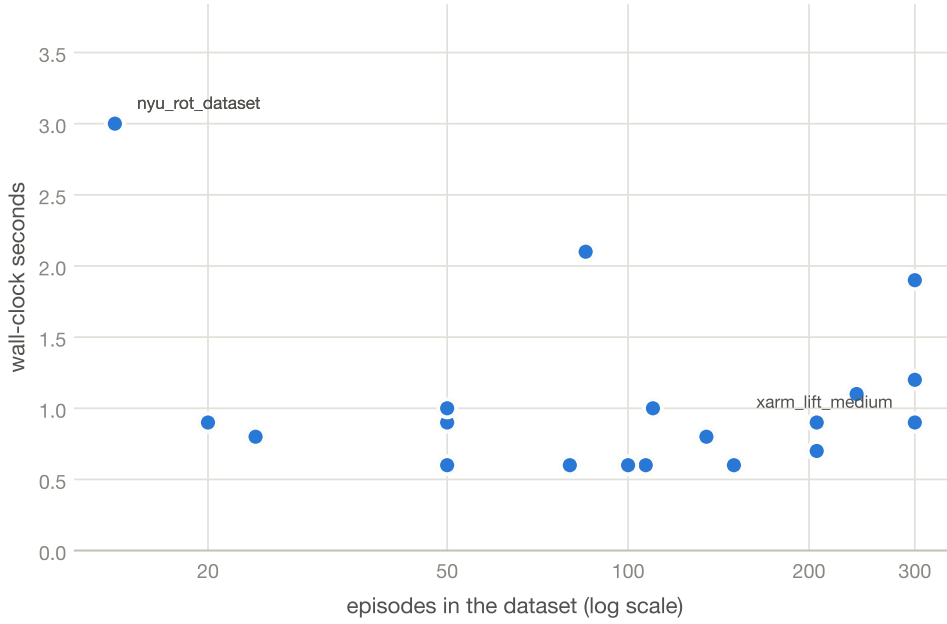


Figure 1: Scan wall-time against dataset size (log x). The relationship is flat: bohrin reads Parquet columns and never decodes video, so per-dataset cost is dominated by fixed per-detector work rather than by frame count.

originates as RLDS/Open-X conversions rather than native LeRobot recordings. Distinguishing these requires the adjudication described in Section 5.

3.4 The nineteen silent detectors

Nineteen of the 46 default detectors produced no finding across 2,631 episodes. This is not evidence that they are broken — several target defects this corpus should not have (`integrity.nan_inf`; the vision family, with decoding disabled; the entire `POLICY↔DATA` family, which stays silent without a checkpoint). It does mean those detectors carry *zero real-data evidence* in this run, and their default severities rest on synthetic fixtures alone. It would be incorrect to describe this run as having validated 46 detectors.

4 Two Detectors Were Correctly Held Back

Bohrin’s registry defines `DEFAULT_EXCLUDED`: detectors that are implemented, tested, and benchmarked, but withheld from a default scan pending recalibration and reachable with `--all`. It currently holds `smoothness.discontinuity_jump` and `integrity.declared_mismatch`, excluded after an earlier run of this same harness measured them reporting HIGH on 70% and 60% of the corpus respectively. Neither appears in any finding in this sweep, confirming the mechanism works.

This is the project’s stated alternative to deleting a noisy detector: measure it, record the number, and gate it. Removing tested, benchmarked code to make a report look cleaner is a regression, not a cleanup.

5 Limitations

These bound every number above and should be read before any of them is quoted.

Fire rate is not precision. Nothing here establishes that a single finding is correct. A 95% fire rate is equally consistent with “95% of public robot datasets have this defect” and with “the threshold is too tight.” No ground-truth labels exist for this corpus and none were constructed. *Every rate reported here*

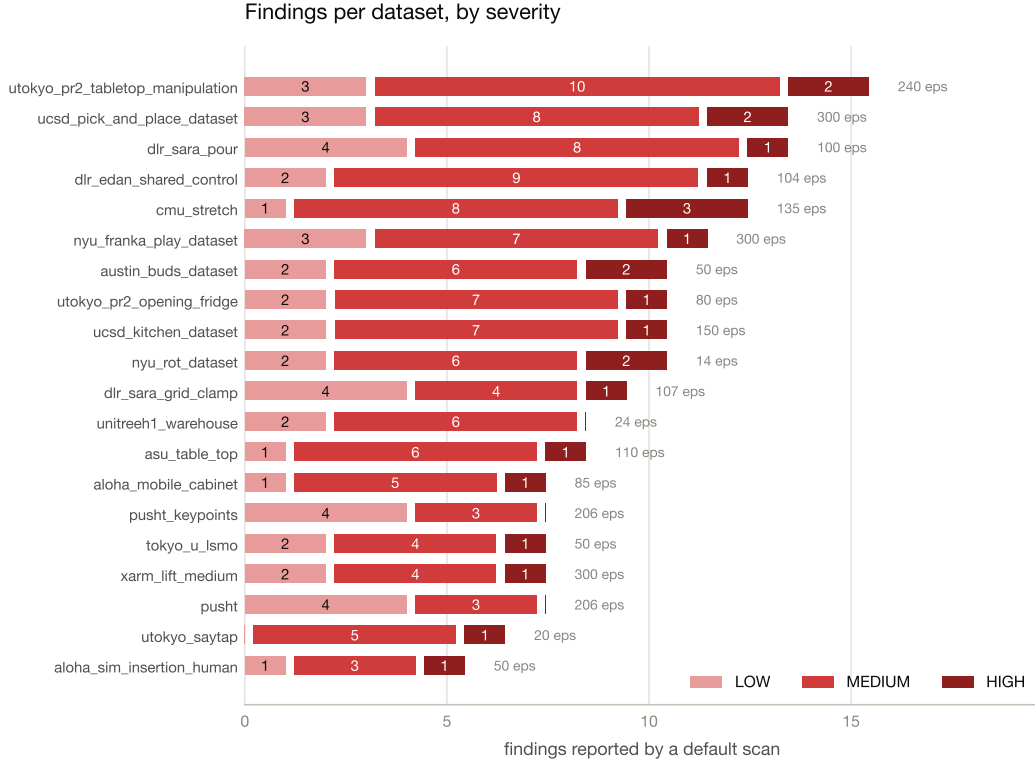


Figure 2: Findings per dataset, split by severity. Every dataset produced findings; 17 of 20 produced at least one HIGH.

is a rate of complaint, not a rate of correctness. The next step is to hand-adjudicate a stratified sample against the underlying Parquet and publish a precision estimate with an explicit N .

Selection bias, in both directions. The datasets were chosen by the project’s own maintainer. They are curated and widely used, which biases toward clean data and makes a high fire rate *more* suspicious — the intended direction. But they are also predominantly 5 Hz Open-X/RLDS conversions, a narrow slice of how robot data is recorded, so conversion-pipeline artifacts are over-represented.

Single machine, single run. One laptop (Apple M4, 16 GB, macOS 15.5, Python 3.10.21), one pass, warm cache. Timings are indicative, not benchmarked: no repeats and no confidence intervals.

Default configuration only. No checkpoint, target, calibration corpus, or vision. The five POLICY $\leftrightarrow$ DATA detectors were inert by construction. A different configuration yields different numbers.

The conformal gate was never active, and public metadata may prevent fixing that. Bohrin’s `--fpr` flag becomes a genuine false-discovery bound only when a calibration corpus covers the dataset’s embodiment — Mondrian conformal prediction, keyed on embodiment as the taxonomy. No corpus ships with the package, so every scan fell back to a robust- z heuristic. Filling that gap meets an obstacle this sweep surfaced incidentally: **18 of the 20 datasets declare `robot_type: "unknown"`**, with only the two ALOHA datasets naming an embodiment. A Mondrian taxonomy keyed on a field that 90% of public data leaves blank collapses to a single wildcard bucket, forfeiting exactly the group-conditional validity the design was chosen for. This is a finding about ecosystem metadata hygiene as much as about the tool.

Warnings observed, not investigated. Several scans emitted `RuntimeWarning: divide by zero / overflow / invalid value from sklearn.utils.extmath` during dynamics fits. No scan failed and results are reported as produced, but these indicate degenerate matrices reaching the ridge solve.

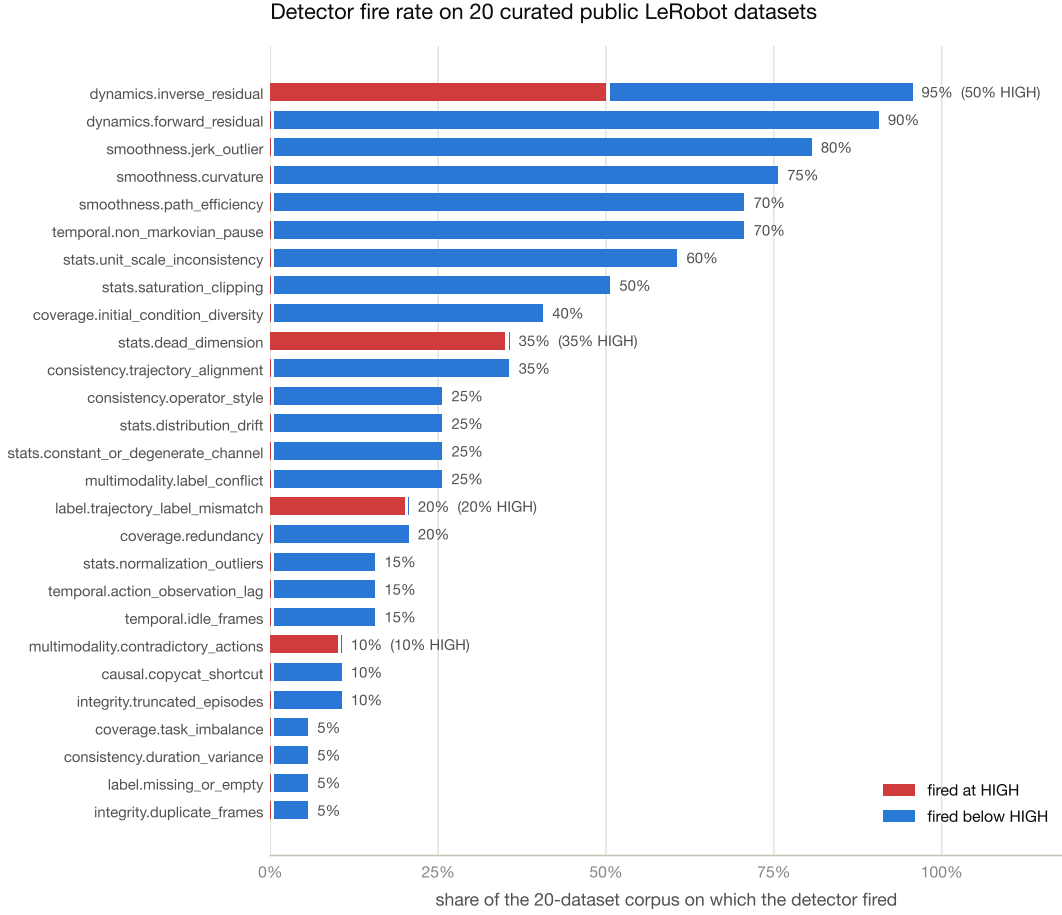


Figure 3: Per-detector fire rate across the 20-dataset corpus, with the HIGH share called out in red. Twenty-seven detectors appear; the other nineteen produced no finding on any dataset.

6 Next Steps

1. Adjudicate `dynamics.inverse_residual` by hand-labelling a stratified sample of its HIGH findings. Either fix the threshold with the evidence, or add it to `DEFAULT_EXCLUDED` with the numbers recorded.
2. Publish a precision estimate with an explicit N . A measured number on 20 adjudicated findings is worth more than an unfalsifiable claim about 48 detectors.
3. Resolve the embodiment-metadata gap before shipping a default calibration corpus — either infer embodiment from action/state dimensionality and control rate, or accept the wildcard bucket and stop claiming group-conditional validity.
4. Trace the `matmul` warnings in the dynamics fits.

Reproducibility

```
git clone https://github.com/prabhu-gopal/bohrin && cd bohrin
git checkout cf39d78
python -m venv .venv && .venv/bin/pip install -e '[dev]'
.venv/bin/python scripts/hub_smokey.py --json sweep.json
```

Raw results, figure-generation code, and this document's source are committed under `benchmarks/2026-08-26-hub-sweep-v0.1.0/`. Every figure derives from `results/sweep.json` alone; the commit-

ted JSON is the authority if a figure and the text ever disagree. Hub dataset revisions are not pinned, so an upstream update will change the numbers.

Environment: Python 3.10.21, numpy 2.2.6, polars 1.44.0, scikit-learn 1.7.2, huggingface-hub 1.28.0, pydantic 2.13.4, rich 15.0.0; macOS 15.5 (Darwin 25.5.0), Apple M4, 16 GB. Figures: matplotlib 3.11.1, generated in a separate environment — matplotlib is a reporting tool and deliberately not a bohrin dependency.